

Balance hídrico en estaciones INIA (2016–2025)

Cómo hicimos el trabajo, paso a paso · Programación Avanzada para Ciencias de Datos

Docente: Marcelo Danessi
Autores: Alejandra Ramírez, Alvaro Barreto, Katerin Gonzalez y Gabriela Flores

Este documento explica, en el orden en que se hizo, todo el proceso para cumplir la consigna de la actividad final: transformar el trabajo del semestre sobre nuestra base de datos en un proyecto reproducible de ciencia de datos. En cada paso se indica qué punto de la consigna cumple.

Paso 1. Leer la consigna y armar la lista de requisitos

Leímos la consigna y separamos lo obligatorio en una lista de control: estructura de carpetas, datos originales separados de los procesados, base procesada generada automáticamente, al menos dos validaciones, funciones propias, automatización con **map**, transformación temporal o indicadores, al menos dos visualizaciones, SQL, una técnica del curso, tablas automáticas, README e informe en Quarto. También anotamos la advertencia del profesor: *no agregar técnicas solo para cumplir una lista*, sino elegir las que tengan sentido para la pregunta.

Paso 2. Revisar el material del curso

Leímos las clases (regresión, clustering, análisis temporal, automatización con funciones y purrr, proyectos reproducibles con Quarto) y los scripts de las actividades anteriores. Así usamos las mismas herramientas y la misma forma de trabajar vistas en clase: **dplyr**, **lubridate**, **purrr**, **slider**, **ggplot2**, **broom**, **DBI** y **Quarto**.

Paso 3. Explorar los datos

Revisamos los tres archivos y el archivo de metadatos que explica cada columna:

- **INIA.csv:** 21.402 registros diarios de 6 estaciones, de 2016 a marzo de 2026.
- **EstacionAgr.csv** y **Departamentos.csv:** catálogos que relacionan cada estación con su departamento.

En esta exploración ya aparecieron los primeros problemas: una estación (Glencore) con muchos huecos, días con temperatura y humedad en 0 al mismo tiempo, humedades mayores a 100 % y el año 2026 incompleto.

Paso 4. Elegir la pregunta de análisis

> **¿Cómo se comportó el balance hídrico (lluvia − evapotranspiración) en las estaciones de INIA entre 2016 y 2025, qué tan fuerte fue la sequía de 2022–2023 y afectó por igual al sur y al norte?**

Como en las actividades anteriores ya se había trabajado la precipitación con regresión y clustering, elegimos una pregunta temporal que aprovecha la misma base. Usamos el balance y no solo la lluvia porque para el campo importa cuánta agua queda disponible.

Paso 5. Organizar el proyecto en carpetas

Cumple con: estructura organizada de carpetas; separación entre datos originales y procesados.

Carpeta o archivo	Para qué sirve
data/raw/	Datos originales. Nunca se modifican.
data/processed/	Datos limpios y base SQLite. Los genera el código.
R/	Funciones propias (funciones.R) y configuración común (configuracion.R).
scripts/	Los pasos del análisis, numerados del 00 al 04.
sql/	Las consultas SQL (consultas.sql).
outputs/	Tablas (CSV) y gráficos (PNG) generados.
report/	Informe Quarto.
run_all.R	Ejecuta todo el proyecto en orden.

Todos los caminos son relativos (por ejemplo *data/raw/INIA.csv*), así el proyecto funciona en cualquier computadora.

Paso 6. Instalar los paquetes (script 00)

00_paquetes.R verifica qué paquetes faltan y los instala en una carpeta del propio proyecto (*r_packages/*), igual que en los ejemplos de clase. Así el proyecto no depende de lo que tenga instalado cada computadora.

Paso 7. Preparar los datos (script 01)

Cumple con: generación automática de una base procesada; al menos dos validaciones; funciones propias; automatización con map.

1. **Leer** los tres archivos originales.
2. **Validar la estructura:** que existan las columnas necesarias y que todas las estaciones de INIA estén en el catálogo. Si algo falta, el programa se detiene con un mensaje claro.
3. **Corregir los nombres:** normalizamos las tildes y corregimos la errata “Tacuaremnó” → “Tacuarembó”.
4. **Unir las tablas** (INIA → estación → departamento) y validar que ningún registro quede sin departamento ni con fecha inválida.
5. **Validar la calidad por estación** con la función propia *diagnosticar_estacion()*, aplicada a las 6 estaciones con **map_dfr()**: fechas duplicadas, días ausentes, ceros falsos, valores fuera de rango, Tmax menor que Tmin, etc. El resultado se guarda como tabla.
6. **Limpiar** con *limpiar_inia()*: los ceros falsos (227 días) y los valores imposibles pasan a NA. No se borra ninguna fila.
7. **Calcular el balance diario** = precipitación − evapotranspiración.
8. **Crear la base mensual** con *resumir_mensual()*, aplicada a cada estación con **group_nest()** + **map()**. Incluye la cobertura de cada mes.
9. **Guardar** las bases procesadas (diaria y mensual) en *data/processed/*.

> **Regla de cobertura:** un mes solo se usa si tiene al menos el 80 % de los días con datos, y un año solo si tiene sus 12 meses válidos. Así quedan afuera, de forma automática y no a mano, el año 2026 (incompleto) y los huecos de Glencore.

Paso 8. Armar la base relacional y las consultas SQL (script 02)

Cumple con: al menos una consulta SQL o interacción con una estructura relacional.

Guardamos los datos en una base **SQLite** con tres tablas relacionadas por claves: *departamentos*, *estaciones* y *mensual*. Validamos con un LEFT JOIN que todas las filas encuentren su estación y su departamento. Las consultas están en *sql/consultas.sql* y usan **JOIN**, **GROUP BY** y **HAVING**:

- los 10 meses más secos, con su departamento;
- el balance anual por estación (solo años completos);
- el balance de 2022–2023 por estación y departamento.

El código ejecuta cada consulta automáticamente con **map** y guarda cada resultado como CSV.

Paso 9. Hacer el análisis temporal (script 03)

Cumple con: transformación temporal e indicadores; técnica del curso (análisis temporal); al menos dos visualizaciones.

- **Anomalías:** cada mes comparado con el promedio de ese mismo mes en la estación.
- **Variación respecto del mes anterior** con **lag()**.
- **Balance acumulado de 12 meses** con **slider**, para detectar sequías largas.
- **Agregación anual**, solo de los años completos.
- **Indicadores de sequía** por estación: el peor acumulado, el mes en que ocurrió y el porcentaje de meses con déficit.

Con eso generamos los gráficos con **ggplot2**: cobertura de datos, mapa de calor de anomalías, acumulado de 12 meses y balance anual, más un gráfico por estación guardado automáticamente con **walk()**.

Paso 10. Ajustar la regresión de tendencia (script 04)

Cumple con: técnica del curso (regresión); automatización con map.

Para saber si la sequía era parte de una tendencia o un episodio, ajustamos una regresión lineal *balance_anual ~ año* para cada estación con **group_nest()** + **map()**. Usamos **possibly()** para que un error en una estación no detenga todo, y una columna de estado que registra qué modelos se ajustaron. Los resultados se extraen con **broom**. En ninguna estación apareció una tendencia significativa.

Paso 11. Generar las tablas de resultados automáticamente

Cumple con: generación automática de tablas o archivos de resultados.

Cada script guarda sus resultados en *outputs/tables/* (12 archivos CSV) y *outputs/figures/*. Creamos la función *exportar_tabla()* para que todas las tablas se guarden redondeadas y sin errores de punto flotante.

Paso 12. Unir todo en un solo comando: run_all.R

run_all.R ejecuta los scripts 00 a 04 en orden, cada uno por separado, y al final genera el informe. También guarda las versiones de R y de los paquetes utilizados (*sessionInfo*).

Paso 13. Escribir el informe Quarto y el README

Cumple con: informe reproducible con Quarto; archivo README.md.

El informe (*report/informe.qmd*) tiene todas las secciones que pide la consigna: pregunta, datos, decisiones de preparación, problemas de calidad, análisis, tablas y gráficos, resultados, limitaciones y conclusiones. No recalcula nada: lee las tablas y gráficos generados por los scripts, y los números del texto se insertan con código, por lo que se actualizan solos si cambian los datos.

El **README.md** explica el objetivo, los datos, la estructura de carpetas, los requisitos, cómo ejecutar el proyecto, las decisiones principales, los resultados y las limitaciones.

Paso 14. Probar la reproducibilidad

Cumple con: reproducibilidad desde los datos originales y el código.

Borramos todo lo generado (datos procesados, base SQL, tablas, gráficos e informe) y ejecutamos **run_all.R**. Se reconstruyó todo sin errores, sin modificar nada a mano. Repetimos esta prueba después de cada cambio importante.

Paso 15. Revisar y verificar los resultados

- Recalculamos los resultados principales con un programa aparte, directo desde los datos crudos y sin usar nuestras funciones. Coincidieron todos.
- Encontramos y corregimos un error propio: los días con lluvia por mes se calculaban mal (daban 0 o 1). No afectaba las conclusiones.
- Revisamos el texto del informe contra los datos y precisamos frases que generalizaban de más.

Problemas que encontramos y cómo los resolvimos

Problema	Cómo lo resolvimos
227 días con temperatura 0 °C y humedad 0 % (dato faltante disfrazado)	Los pasamos a NA
12 humedades fuera de 0–100 % y 1 evaporación negativa	Los pasamos a NA, sin borrar filas
Glencore sin datos en 2019 y gran parte de 2020	Regla automática de cobertura (80 %)
2026 incompleto (los datos llegan hasta marzo)	La misma regla lo excluye sola
Nombre mal escrito (“Tacuaremnó”) y tildes guardadas como dos caracteres	Normalizamos el texto y corregimos el nombre después de unir las tablas
Un gráfico unía con una recta los años sin datos	Cortamos la línea en los huecos
Error propio en el conteo de días con lluvia	Lo detectamos al revisar el código y lo corregimos

Resumen: cada requisito de la consigna y dónde se cumple

Requisito de la consigna	Dónde se cumple
Estructura organizada de carpetas	data/, R/, scripts/, sql/, outputs/, report/
Datos originales separados de los procesados	data/raw/ y data/processed/
Base procesada generada automáticamente	01_preparar_datos.R
Al menos dos validaciones	Columnas, catálogo, join, calidad por estación, claves en SQL
Funciones propias	R/funciones.R (13 funciones)
Automatización con map	map, map_dfr, group_nest + map, possibly, walk
Transformación temporal o indicadores	Mensual, anual, anomalías, lag, acumulado 12 meses
Al menos dos visualizaciones	5 gráficos generales + 1 por estación
Consulta SQL o estructura relacional	SQLite con 3 tablas y 3 consultas (02_base_relacional.R)
Técnica del curso	Análisis temporal y regresión de tendencia
Tablas o archivos de resultados automáticos	12 CSV en outputs/tables/
README.md	Raíz del proyecto
Informe reproducible con Quarto	report/informe.qmd → informe.html
Reproducibilidad	run_all.R reconstruye todo desde data/raw/

> ***En una frase:*** *partimos de los datos originales, los validamos y limpiamos con funciones propias, los automatizamos para las 6 estaciones con map, los analizamos en el tiempo, con SQL y con regresión, y todo —tablas, gráficos e informe— se reconstruye con un solo comando.*